

Sisteme de operare

Laborator 7

Semestrul I 2025-2026

Laborator 7

- comunicare inter-procese (IPC)
- sisteme message passing
 - anonymous pipes
 - FIFOs (named pipes)

Pipe

- unidirectionale (half-duplex)
- nu pot fi folosite decat intre procese cu acelasi stramos
 - scenariu tipic: procesul parinte creaza un *pipe*, si apoi apeleaza *fork*

```
int pipe(int fd[2]);
```

- fd[0] deschis pentru citire, fd[1] pentru scriere (iesirea lui fd[1] este intrarea lui fd[0])
- pt. citire/scriere din *pipe* se folosesc apelurile *read/write*

```
read(fd[0], buf, len);
```

```
write(fd[1], buf, len);
```

- dupa *fork*, copilul si parintele inchid unul dintre capetele de acces ale pipe-ului, in functie de directia de comunicare dorita
 - ex:


```
close(fd[0]);          // parinte
close(fd[1]);          // copil
```

=> directia de comunicare este parinte -> copil

Pipe (cont.)

- secventa de apel uzuala

```
#include <unistd.h>

int main(int argc, char* argv[]) {
    int fd[2];
    if(pipe(fd) < 0)
        perror("pipe"), exit(1);
    if(!fork())
        close(fd[1]); // copilul inchide capatul sau de scriere
    else
        close(fd[0]); // parintele inchide capatul sau de citire
}
```

Semantica operatiilor cu pipe-uri

- citirea dintr-un *pipe* gol blocheaza procesul
- scrierea intr-un *pipe* plin blocheaza procesul pana cand s-a citit suficient pentru ca scrierea sa se poata termina
- I/O non-blocant se seteaza cu ajutorul *fcntl*
- daca toate capetele de scriere ale unui *pipe* sunt inchise, *read* intoarce 0 octeti (EOF)
- daca toate capetele de citire ale unui *pipe* sunt inchise, *write* determina trimiterea unui semnal SIGPIPE catre procesul apelant
 - daca se ignora semnalul, *write* intoarce -1 cu *errno* = *EPIPE*
- nu se poate apela *lseek* pe un *pipe*
- scrierile mai mici de PIPE_BUF octeti sunt atomice, mai mult de PIPE_BUF pot fi interleaved cu scrierile altor procese

FIFOs (named pipes)

- *pipe* creaza un canal de comunicatie intre procese cu un stramos comun care a creat *pipe*-ul
- fisierele de tip FIFO permit IPC intre procese diferite
- apelul *stat/fstat* pe un fisier de tip FIFO indica in *st_mode* ca e fisier special (se poate folosi macro-ul *S_ISFIFO*)

int mkfifo(const char pathname, mode_t mode);*

- parametrul *mode* are aceeasi semnificatie ca la *open*, ex:

mkfifo("myfifo", 0644);

- dupa ce s-a creat fisierul FIFO, se deschide cu *open*, in mod uzual
- toate operatiile uzuale de I/O cu fisiere (*open*, *close*, *read*, *write*, *unlink*, etc) functioneaza si cu fisiere FIFO

FIFOs (cont.)

- la deschiderea unui FIFO
 - fara O_NONBLOCK: *open* cu O_RDONLY blocheaza procesul pana cand alt proces deschide FIFO-ul pt scriere; similar pt *open* cu O_WRONLY, procesul se blocheaza pana cand un alt proces deschide FIFO-ul pt citire
 - cu O_NONBLOCK: *open* cu O_RDONLY se intoarce imediat, in schimb cu O_WRONLY rezulta eroare cu *errno* = *ENXIO* daca nu exista nici un proces care a deschis FIFO pt citire
- citirile/scrierile au aceeasi semantica ca la *pipe*
- scrierile multiple concurente sunt atomice cata vreme scriu mai putin de PIPE_BUF octeti
- scenarii tipice de utilizare FIFOs
 - faciliteaza shell-ului evitarea folosirii fisierelor temporare cand transfera date de la un pipeline la altul
 - aplicatii client-server

mknod

- traditional, sistemele UNIX au creat FIFOs cu comanda/apelul sistem *mknod*
 - metoda generala de a crea fisiere in Unix:
 - fisiere obisnuite: S_IFREG
 - device-uri caracter S_IFCHR
 - device-uri block S_IFBLK
 - FIFOs S_IFIFO
 - Unix sockets S_IFSOCK

int mknod(const char pathname, mode_t mode, dev_t dev);*

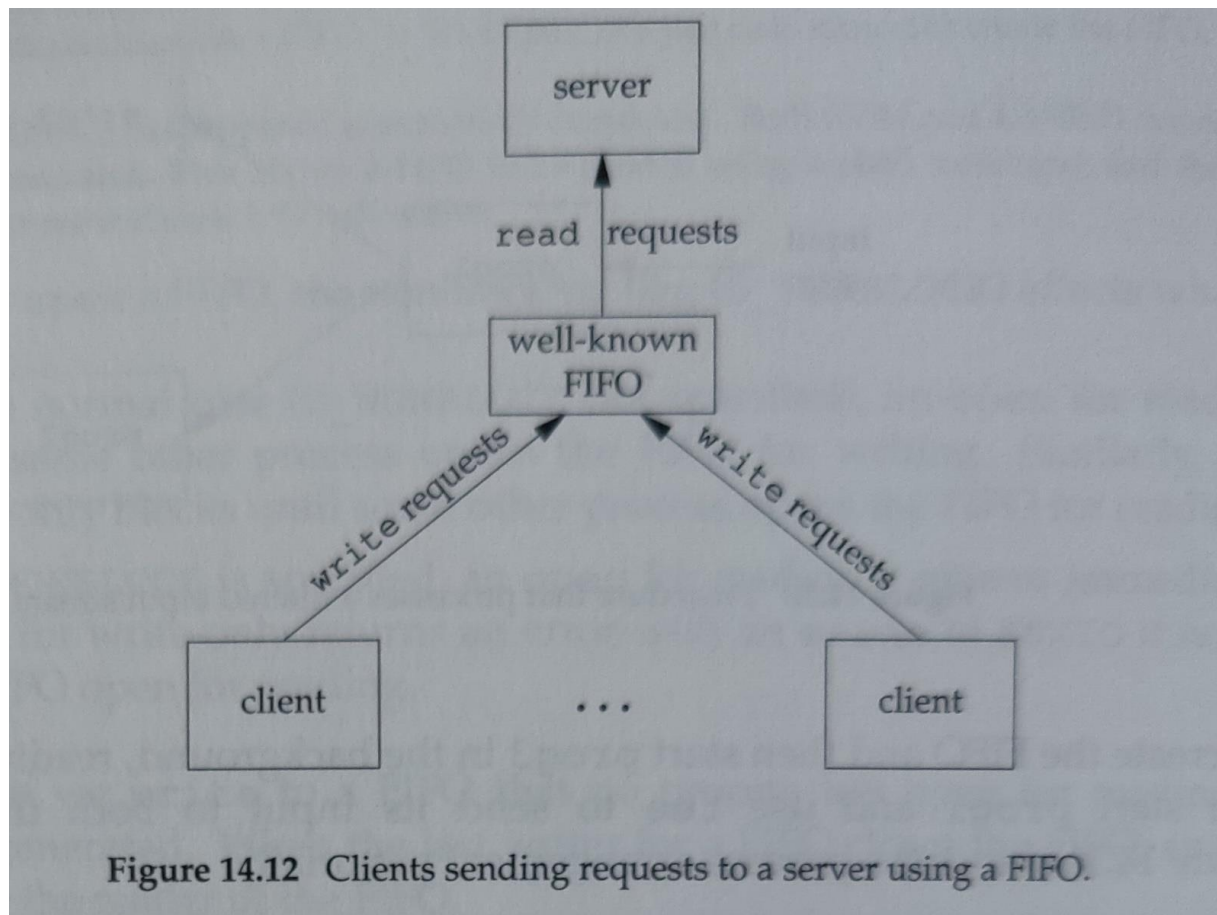
- in particular, pt FIFO

mknod("myfifo", S_IFIFO | 0644, 0);

Arhitectura client-server cu FIFO

Probleme:

- cererile client trebuie sa fie mai mici de PIPE_BUF octeti pt. a fi citite atomic (fara intretesere intre scrierile clientilor)
- cum trimitem raspunsurile serverului catre fiecare client?
 - nu se poate folosi un singur FIFO pt toti clientii
 - un client nu stie cand sa-si citeasca propriul raspuns, si nu pe ale celorlalti clienti
- ambele probleme deriva din faptul ca nu exista canal de comunicatie separat intre client si server



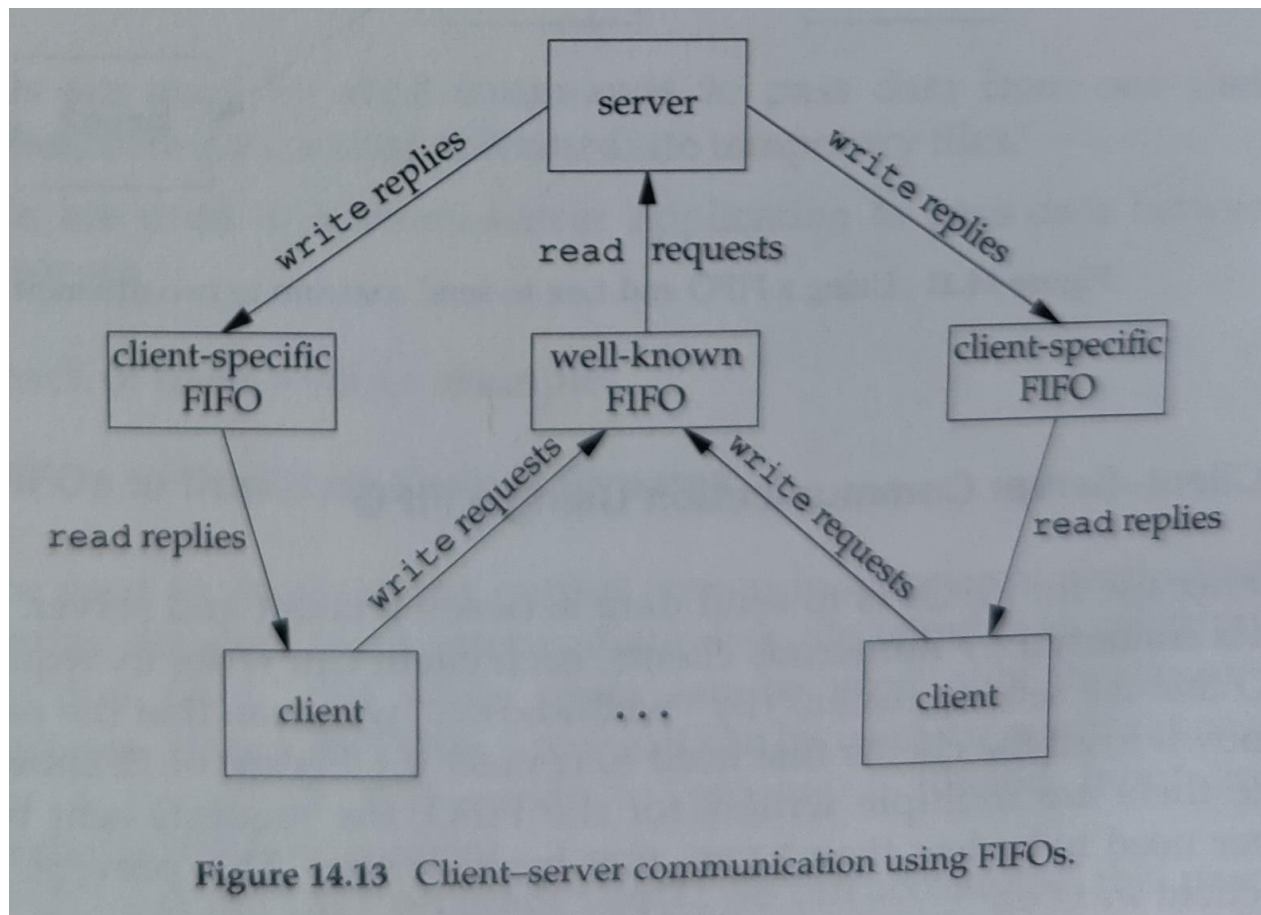
Comunicare client-server cu FIFO

Solutie posibila:

- fiecare client isi trimite cererea insotita de PID-ul sau
- serverul creeaza un FIFO de raspuns pt fiecare client folosind un nume care incorporeaza PID-ul client
 - ex: /tmp/client.XXXXX unde XXXXX = PID client

Probleme:

- clientul se defecteaza, serverul nu are cum sa afle => raman FIFO-uri in sistem
- serverul trebuie sa prinda SIGPIPE daca clientul termina inainte sa citeasca raspunsul serverului
- daca serverul deschide FIFO-ul RO, cand nr de clienti scade la 0, serverul citeste EOF
 - solutie: serverul deschide FIFO *read-write*



Sisteme de operare

Laborator 7

Comunicare inter-procese, pipe-uri

1. Creati un nou subdirector *lab7/* in structura de directoare a laboratorului creata anterior (*SO/laborator*) si subdirectoarele aferente *doc src* si *bin*. Nu uitati sa actualizati variabila de mediu *PATH* pentru a include directorul *SO/laborator/lab7/bin*.

2. Scrieti un program C **pipe.c** care creaza un *pipe* intre un proces parinte si un copil. Parintele scrie mesajul "*hello world\n*" in *pipe* si asteapta sa se termine procesul copil. Copilul citeste mesajul trimis de parinte in *pipe* si il afiseaza pe ecran. Primul lucru pe care il fac atat parintele cat si copilul dupa crearea *pipe*-ului este sa inchida partea de *pipe* pe care nu o folosesc la comunicarea dintre ei.

3. Scrieti un program C **pipe2.c** care modifica programul de la punctul 2 a.i. :

- procesul copil nu inchide capatul sau de scriere in *pipe*
- dupa ce a citit din *pipe* mesajul scris de procesul parinte, copilul scrie inapoi in *pipe* mesajul "*done\n*" si se termina
- dupa ce a scris in *pipe* mesajul "*hello world\n*" pentru copil, parintele asteapta sa citeasca din *pipe* mesajul "*done\n*" scris de copil si apoi il afiseaza pe ecran.

Ce se intampla? Cum explicati situatia?

4. Scrieti un program C **sigpipe.c** care creaza un *pipe* intre un proces parinte si un copil. Copilul inchide capatul sau de citire din *pipe*, simuleaza executia unui job folosind functia *sleep* pentru a dormi 1 secunda si se incheie. Parintele inchide si el capatul sau de citire din *pipe*, apeleaza *sleep* pentru 2 secunde dupa care scrie mesajul "*hello world\n*" in *pipe* si asteapta sa se termine procesul copil.

Ce se intampla? Dar daca programul prinde SIGPIPE ce se intampla?

5. Scrieti un program C **vfork-pipe.c** care creeaza un proces copil cu ajutorul apelului sistem *vfork* si creeaza un *pipe* intre cele doua procese. Procesul copil citeste un caracter din *pipe* si il afiseaza pe ecran impreuna cu PID-ul sau. Apoi incheie executia cu succes (cod zero). Procesul parinte scrie caracterul 'b' in *pipe* dupa care incheie executia.

Ce se intampla? Cum explicati situatia?

6. Scrieti un program C **ls-more.c** care emuleaza urmatoarea comanda shell:

```
$ ls -l | less
```

Programul creeaza in procesul parinte un *pipe* si doua procese copil, primul responsabil de executia comenzii *ls -l*, iar cel de-al doilea pentru comanda *less* (alegeti cea mai potrivita varianta de apel *exec* folosind exemplul comenzii emulate). Dupa ce a creat procesele copil, procesul parinte se blocheaza asteptand terminarea lor si recupereaza starea lor de terminare.

Ce se intampla daca parintele nu inchide capatul sau de scriere?

7. Rescrieti programul C de mai sus cu numele **ls-more2.c** pentru a emula urmatoarea comanda shell:

```
$ ls -l | tee | ... | tee | less
```

unde comanda *tee* poate aparea de **n** ori in pipeline. **n** trebuie sa fie parametru al programului in linie de comanda ca mai jos, de exemplu:

```
$ ls-more2 3
```

Comanda de mai sus trebuie sa fie echivalenta cu urmatoarea comanda:

```
$ ls -l | tee | tee | tee | less
```

Indicatii: Avand in vedere numarul variabil de pipe-uri din linia de comanda, programul vostru trebuie sa tokenizeze linia de comanda, spre deosebire de programul anterior. De asemenea, aveti grija cum gestionati capetele fiecarui pipe din linia de comanda.

8. Scrieti un program C **my-mkfifo.c** care foloseste apelul *mkfifo* pentru a crea un *named pipe* (fisier de tip *FIFO*) cu numele dat ca argument in linie de comanda. Programul are efectul urmatoarelor comenzi shell:

```
$ mknod <nume fifo> p
```

sau

```
$ mkfifo <nume fifo>
```

Folositi comanda *ls -l* pentru a identifica daca fisierul creat are attribute de fisier *FIFO*.

```
ex:  $ mknod myfifo p
      $ ls -l myfifo
```

Apoi, scrieti doua programe C **fifo-echod.c** si **fifo-echo.c** care scriu si citesc dintr-un fisier creat cu una dintre comenzile anterioare (fie cea rezultata prin compilarea programului **my-mkfifo.c**, fie prin comenzile shell de mai sus). **fifo-echod.c** deschide pentru citire si scriere fisierul *FIFO* primit ca parametru in linie de comanda si citeste in bucla caractere din fisierul *FIFO*. Dupa fiecare string de caractere citit din *FIFO*, **fifo-echod.c** trimite inapoi in *FIFO* string-ul citit (i.e., are functionalitate de serviciu de echo). Programul **fifo-echod.c** se lanseaza in background ca in exemplul de mai jos si asteapta sa primeasca mesaje in fisierul *FIFO*:

```
$ gcc -o fifo-echod fifo-echod.c
$ fifo-echod myfifo &
$ jobs
```

Programul **fifo-echo.c** deschide pentru citire si scriere fisierul *FIFO* primit ca parametru in linie de comanda si citeste in bucla caractere de la tastatura, de ex cu functia *fgets*. Dupa fiecare citire, programul scrie in *FIFO* caracterele citite de la tastatura, asteapta ecoul de la *echod* (i.e. citeste din *FIFO* ceea ce scrie *echod*) si tipareste caracterele pe ecran. Programul ruleaza la infinit pana cand e oprit cu Ctrl-C.

```
$ gcc -o fifo-echo fifo-echo.c
$ fifo-echo myfifo
```

Indicatie: pentru a scrie mai repede programele **fifo-echod.c** si **fifo-echo.c** refolositi codul de la programele **ping.c** si **pong.c** din laboratorul 3.

9. Modificati programul **my-shell.c** din laboratoarele trecute pentru a implementa pipeline-uri. Mai exact, shell-ul trebuie sa fie capabil sa execute comenzi de tipul celor de mai jos:

```
$ my-shell
my-shell> ls -l | more
my-shell> cat /etc/hosts | tee my-hosts-copy | wc -l
my-shell> cat /etc/passwd | grep `whoami` | wc -c
my-shell> Ctrl-d
$
```

Indicatii: Folositi codul de la exercitiul 7 pentru a gestiona pipeline-urile. De asemenea, observati ca pentru a gestiona atat pipeline-uri cat si comenzi obisnuite, **my-shell** are nevoie de tokenizarea liniei de comanda pe doua niveluri, intai obtine token-rile individuale din pipeline, pe care apoi trebuie sa le lege intre ele ca la exercitiul 7, dar si sa sub-tokenizeze fiecare comanda individuala din pipeline pentru a putea executa comanda respectiva folosind *exec*. In acest scop trebuie sa folositi varianta reentranta a functiei *strtok*, si anume *strtok_r*.